

[BE-4] Database Table Relation dengan GORM

1. Persiapan

Untuk mengikuti pembelajaran ini kamu perlu menyiapkan:

- Bahasa **Go Lang** (<https://go.dev/>).
- **Git** untuk clone starter proyek (<https://git-scm.com/>).
- **VS Code** untuk text editor, atau editor kesukaanmu (<https://code.visualstudio.com/>). Pastikan menginstall Plugin Go Lang.
- **Postman** untuk test API yang kita buat nanti (<https://www.postman.com/>).

2. Penjelasan Database Relation

💡 **Apa itu Relasi dalam Database?**

Relasi dalam database adalah hubungan antara tabel berdasarkan **Foreign Key**.

📌 **Jenis-jenis Relasi dalam Database:**

🟢 One-to-One (1:1)

- **Deskripsi:** Satu baris di tabel A hanya boleh berhubungan dengan satu baris di tabel B.
- **Contoh:**
 - Satu **User** hanya boleh memiliki **satu** buku favorit.
 - Satu **karyawan** hanya memiliki **satu kartu identitas**.
- **Struktur Relasi:**

User (1) ↔ (1) FavoriteBook

🟢 One-to-Many (1:M)

- **Deskripsi:** Satu baris di tabel A dapat memiliki banyak baris terkait di tabel B.
- **Contoh:**
 - Satu **User** bisa menulis banyak **Book**.
 - Satu **kategori** bisa memiliki banyak **produk**.
- **Struktur Relasi:**

User (1) ↔ (M) Book

🟢 Many-to-Many (M:M)

- **Deskripsi:** Banyak baris di tabel A dapat memiliki banyak hubungan dengan tabel B.
- **Contoh:**
 - Seorang **User** bisa menambahkan banyak **Book** ke wishlist.
 - Seorang **siswa** bisa mengambil banyak **mata kuliah**, dan satu **mata kuliah** bisa diambil banyak siswa.
- **Struktur Relasi:**

User (M) ↔ (M) Book (melalui tabel pivot: user_books)

3. Implementasi relasi One-to-One: User & FavoriteBook

- a. Sebelumnya, kita akan menggunakan endpoint `/login` yang sudah kita buat sebelumnya untuk mendapatkan data user, kita hanya cukup menambahkan data user pada return di controller login

```
return c.JSON(fiber.Map{
    "message": "Login sukses",
    "user":    user, //tambahkan ini
})
```

- b. Oke, mari masuk ke implementasi relasi one-to-one, kita anggap bahwa setiap user akan punya tepat 1 buku favorit. Maka kita bisa tambahkan line berikut di struct `User` :

```
FavoriteBook *Book `gorm:"foreignKey:FavoriteByUserId;constraint:OnDelete:SET NULL;" json:"favorite_e_book"``
```

- c. Lalu tambahkan line berikut di struct `Book` :

```
FavoriteByUserId *uuid.UUID `gorm:"type:uuid" json:"favorite_by_user_id"``
```

- d. Terakhir, kita perlu preload saat mendapatkan data user di `user.repository.impl.go` :

```
if err := db.DB.Preload("FavoriteBook").First(user, "username = ?", username).Error;
```

- e. Jalankan projek dan test.

4. Implementasi relasi One-to-Many: User & Book

Dalam kasus ini, asumsikan bahwa setiap user bisa memiliki banyak buku, artinya, ketika kita menambahkan buku maka buku tersebut ditandai sebagai milik dari si user tersebut. Dari sini, kita akan update controller `AddBook` untuk menambahkan properti `UserId` sebagai penunjuk kepemilikan (bisa dibaca: buku ini adalah milik/dibuat oleh user yang punya id xxx).

- a. Tambahkan line berikut di struct `User` :

```
Books []Book `gorm:"foreignKey:UserId;constraint:OnDelete:SET NULL;" json:"books"``
```

- b. Lalu tambahkan line berikut di struct `Book` :

```
UserId uuid.UUID `gorm:"type:uuid" json:"user_id"``
```

- c. Jangan lupa tambahkan `Preload("Books")` di `user.repository.impl.go` .

```
if err := db.DB.Preload("FavoriteBook").Preload("Books").First(user, "username = ?", username).Error;
```

- d. Sekarang, kita perlu membuat utility baru untuk mengambil user_id dari session yang sedang berjalan. Nantinya, user_id ini kita masukkan setiap kali user menambahkan buku. Di `session.util.go` tambahkan function berikut:

```
func GetIdFromSession(c *fiber.Ctx) uuid.UUID {
    sess := c.Locals("session").(*session.Session)

    id := sess.Get("user_id").(string)
    return uuid.MustParse(id)
}
```

- e. Lalu panggil function dan masukkan nilai `user_id` ke objek book sebelum disimpan ke database dalam function `AddBook` :

```
id := utils.GetIdFromSession(c)
book.UserId = id
```

f. Jalankan projek dan test, cobalah untuk login dan menambahkan beberapa buku.

5. Implementasi relasi Many-to-Many: User & Wishlist

Dalam kasus **Many-to-Many** ini:

✔ Seorang **User** bisa memiliki banyak **Book** dalam wishlist

✔ Sebuah **Book** juga bisa masuk ke wishlist banyak User

Database relasional tidak bisa langsung menyimpan hubungan ini dalam satu tabel, jadi kita butuh **Pivot Table** (tabel perantara) untuk menyimpan hubungan tersebut yang menyimpan id dari kedua objek yang saling berelasi, dalam hal ini `user_id` dan `book_id`.

a. Tambahkan line berikut di struct `User`:

```
WishlistBooks []*Book `gorm:"many2many:user_book;" json:"wishlist_books"``
```

b. Lalu tambahkan line berikut di struct `Book`:

```
InterestedUsers []*User `gorm:"many2many:user_book;" json:"interested_users"``
```

Nantinya, gorm akan membuatkan otomatis table `user_book` sebagai pivot table.

c. Tambahkan `Preload("WishlistBooks")` di `user.repository.impl.go`

```
if err := db.DB.Preload("FavoriteBook").Preload("Books").Preload("WishlistBooks ").First(user, "username = ?", username).Error; err != nil
```

d. Tambahkan `Preload("InterestedUsers")` di `book.repository.impl.go`

```
// GetBookByID implements BookRepository.
func (b bookRepositoryImplPostgres) GetBookByID(id string) (*model.Book, error) {
    book := new(model.Book)
    if err := db.DB.Preload("InterestedUsers").First(book, "id = ?", id).Error; err != nil {
        return nil, err
    }
    return book, nil
}

// GetBooks implements BookRepository.
func (b bookRepositoryImplPostgres) GetBooks() ([]model.Book, error) {
    books := new([]model.Book)
    if err := db.DB.Preload("InterestedUsers").Find(books).Error; err != nil {
        return nil, err
    }
    return books, nil
}
```

e. Jalankan projek dan test, cobalah untuk menambah beberapa relasi di table `user_book`.